

TIER Terminal Commands Cheatsheet

Quick reference for the most common server and analysis commands.

Server details:

- IP: `204.168.166.9`
- SSH user: `root`
- App directory: `/srv/shiny-server/tier-plus/`
- Log database: `/var/log/tier/usage.sqlite`
- Default local SQLite copy: `~/Desktop/tier_usage.sqlite`

⚠ Commands marked **DESTRUCTIVE** delete data permanently. Read twice before running.

1. Connect to the server

From your Mac terminal:

```
bash
ssh root@204.168.166.9
```

You'll be prompted for the root password (unless you've set up SSH keys).

To exit the server back to your local terminal:

```
bash
exit
```

2. Restart shiny-server

After deploying new code or changing config. Run **on the server** (after SSHing in):

```
bash
sudo systemctl restart shiny-server
```

Important: restarting alone doesn't load the app — it loads on the first request. After restarting, visit `http://204.168.166.9:3838/tier-plus/` in your browser to trigger the load.

Check shiny-server status:

```
bash
```

```
sudo systemctl status shiny-server
```

Press **(q)** to exit the status view.

3. Deploy updated files from your Mac to the server

The clean way — a single **(rsync)** of your **(app_deployment/)** directory. Run **on your Mac**.

```
bash

rsync -avz --checksum \
  "/Users/adenoo/OneDrive/AMC/HIV Prioritization tool/HIV_prioritization_tool/code/"
root@204.168.166.9:/srv/shiny-server/tier-plus/
```

What this does:

- **(-a)** archive mode (preserves perms/timestamps)
- **(-v)** verbose (shows what's transferred)
- **(-z)** compress in transit
- **(--checksum)** compares by content hash, not just timestamp+size — catches changes that mtime might miss
- **Trailing slash on source** — copies the *contents* of **(app_deployment/)** into **(/srv/shiny-server/tier-plus/)**, not the directory itself
- Quoted source path because it contains spaces

The command is idempotent — running it twice does nothing the second time (rsync only transfers what changed).

What lives in **(app_deployment/)**

Anything in this folder gets pushed. Confirm these are present locally before deploy:

- **(app.R)** (the renamed interface file — shiny-server expects this filename)
- **(usage_logger.R)**
- **(app_logic.R)** (renamed logic file)
- **(basic_hiv_data.csv)** (if the app loads it)
- Any static assets (e.g. **(www/IAS-logo.png)**)

Files NOT in **(app_deployment/)** are not pushed — keep this folder as the single source of truth for what's deployed.

After any deploy

```
bash
```

```
ssh root@204.168.166.9 "sudo systemctl restart shiny-server"
```

Then visit (`http://204.168.166.9:3838/tier-plus/`) in your browser to trigger app reload.

One-line deploy + restart

If you want both in a single command:

```
bash

rsync -avz --checksum \
  "/Users/adenoo/OneDrive/AMC/HIV Prioritization tool/HIV_prioritization_tool/code/
  root@204.168.166.9:/srv/shiny-server/tier-plus/ && \
ssh root@204.168.166.9 "sudo systemctl restart shiny-server"
```

The (`&&`) runs the restart only if the rsync succeeds.

What NOT to add

- (`--delete`): would delete server files not present locally. Dangerous — a local accidental deletion would wipe production. Stick with the current add/update-only behaviour.
- **Adding more sources without trailing slashes**: changes rsync's behaviour from "copy contents" to "copy the directory itself". Easy to get wrong.

Bonus: own files as shiny user (optional)

By default, rsync preserves the local UID/GID, so files arrive owned by your Mac user (`501 staff`). The shiny user can still read them (perms 644), but ownership looks unusual in (`ls -l`).

To force shiny ownership:

```
bash

rsync -avz --checksum --chown=shiny:shiny \
  "/Users/adenoo/OneDrive/AMC/HIV Prioritization tool/HIV_prioritization_tool/code/
  root@204.168.166.9:/srv/shiny-server/tier-plus/
```

This requires SSHing as root (which you do), and only affects display in (`ls -l`). Functionally equivalent.

4. Pull the usage log SQLite from the server to your Mac

Run on your Mac. Use either:

(`scp`) (simple)

```
bash
```

```
scp root@204.168.166.9:/var/log/tier/usage.sqlite ~/Desktop/tier_usage.sqlite
```

rsync (preserves a meaningful timestamp if you re-run repeatedly)

```
bash
```

```
rsync -avz root@204.168.166.9:/var/log/tier/usage.sqlite \  
~/Desktop/tier_usage.sqlite
```

Versioned pull (keeps history)

If you want a dated copy each time so you don't overwrite previous snapshots:

```
bash
```

```
scp root@204.168.166.9:/var/log/tier/usage.sqlite \  
~/Desktop/tier_usage_$(date +%Y%m%d).sqlite
```

The `$(date +%Y%m%d)` becomes today's date (e.g. `20260610`), so the local file is `tier_usage_20260610.sqlite`.

5. Inspect the usage log on the server (without pulling)

Quick checks via `sqlite3`. Run on the server.

```
bash
```

```
# Total row count
sudo sqlite3 /var/log/tier/usage.sqlite "SELECT COUNT(*) FROM events;"

# Row count by event type
sudo sqlite3 /var/log/tier/usage.sqlite \
  "SELECT event_type, COUNT(*) FROM events GROUP BY event_type;"

# The 5 most recent rows
sudo sqlite3 /var/log/tier/usage.sqlite \
  "SELECT event_id, event_type, user_country, model_country, timestamp
  FROM events ORDER BY event_id DESC LIMIT 5;"

# Country breakdown for user_country (where they accessed FROM)
sudo sqlite3 /var/log/tier/usage.sqlite \
  "SELECT user_country, COUNT(*) FROM events
  WHERE event_type='results_view' GROUP BY user_country ORDER BY 2 DESC;"

# Country breakdown for model_country (what they modelled)
sudo sqlite3 /var/log/tier/usage.sqlite \
  "SELECT model_country, COUNT(*) FROM events
  WHERE event_type='results_view' GROUP BY model_country ORDER BY 2 DESC;"
```

If `sqlite3` isn't installed:

```
bash

sudo apt install sqlite3 -y
```

6. ⚠️ DESTRUCTIVE — Clear or reset the usage log

These delete data permanently. There's no undo.

Backup first (always do this before destructive operations)

```
bash

sudo cp /var/log/tier/usage.sqlite \
  /var/log/tier/usage_backup_$(date +%Y%m%d_%H%M%S).sqlite
```

Delete all rows, keep the schema (recommended)

```
bash

sudo sqlite3 /var/log/tier/usage.sqlite "DELETE FROM events;"
```

Verify:

```
bash
sudo sqlite3 /var/log/tier/usage.sqlite "SELECT COUNT(*) FROM events;"
```

Should return `0`. The app keeps logging into the same file — schema unchanged.

Reclaim disk space after deleting rows (optional)

SQLite doesn't shrink the file on delete. Run this to compact it:

```
bash
sudo sqlite3 /var/log/tier/usage.sqlite "VACUUM;"
```

⚠ Delete the file entirely (stronger — only if you know you want this)

The logger will recreate the file (and schema) on the next session:

```
bash
sudo rm /var/log/tier/usage.sqlite
sudo systemctl restart shiny-server
```

Then visit the app once to trigger file recreation:

```
bash
sudo ls -lh /var/log/tier/usage.sqlite
```

7. Inspect shiny-server logs (for debugging)

Each app session writes a log file in `/var/log/shiny-server/`. Run **on the server**.

```
bash

# Most recent log files (newest first)
sudo ls -lt /var/log/shiny-server/ | head -5

# Read the newest log file (replace filename with whatever ls shows)
sudo cat /var/log/shiny-server/tier-plus-shiny-YYYYMMDD-HHMMSS-NNNNN.log

# Search for specific terms across all logs
sudo grep "usage_logger" /var/log/shiny-server/*.log
sudo grep -i "error" /var/log/shiny-server/*.log | tail -20
```

The system-level shiny-server log (rarely useful for app debugging):

```
bash

sudo tail -100 /var/log/shiny-server.log
```

8. Generate the usage report (on your Mac)

After pulling the SQLite. In RStudio:

```
r

source("~/path/to/analyse_usage.R")
generate_report("~/Desktop/tier_usage.sqlite")
```

Or from a Mac terminal:

```
bash

Rscript ~/path/to/analyse_usage.R ~/Desktop/tier_usage.sqlite
```

Combined one-liner that pulls a fresh SQLite then generates the report:

```
bash

scp root@204.168.166.9:/var/log/tier/usage.sqlite ~/Desktop/tier_usage.sqlite && \
Rscript ~/path/to/analyse_usage.R ~/Desktop/tier_usage.sqlite
```

9. Install R packages on the server

When the logger needs new packages (rare — current setup only needs DBI/RSQLite/digest/jsonlite/httr). Run **on the server**.

```
bash

# Install system-wide as root so the shiny user can see them
sudo su - -c "R -e \"install.packages(c('PKG1', 'PKG2'), repos='https://cran.rstudio.com/)'

# Verify a package is installed
sudo su - -c "R -e \"requireNamespace('PKG1', quietly=TRUE)\""
```

10. Check what's listening on which port

Useful for verifying shiny-server is actually running:

```
bash
sudo netstat -tlnp | grep -E ":80|:443|:3838"
```

Should show (:::3838) (or 0.0.0.0:3838) with shiny-serve as the process if shiny-server is publicly reachable on port 3838.

11. Avoiding having to type passwords (optional, do once)

If you're tired of typing the root password on every SSH/SCP/rsync, set up SSH key authentication.

Run on your Mac:

```
bash
# Generate a key pair (if you don't already have one)
ssh-keygen -t ed25519

# Copy your public key to the server
ssh-copy-id root@204.168.166.9
```

After this, ssh, scp, and rsync all stop prompting for passwords.

Quick-reference cheat row

Action	Command
SSH to server	ssh root@204.168.166.9
Restart shiny-server	sudo systemctl restart shiny-server
Pull SQLite to Mac	scp root@204.168.166.9:/var/log/tier/usage.sqlite ~/Desktop/tier_usage.sqlite
Deploy app_deployment/	rsync -avz --checksum "/path/to/app_deployment/" root@204.168.166.9:/srv/shiny-server/tier-plus/
Count log rows	sudo sqlite3 /var/log/tier/usage.sqlite "SELECT COUNT(*) FROM events;"

Action	Command
⚠ Clear log rows	<code>sudo sqlite3 /var/log/tier/usage.sqlite "DELETE FROM events;"</code>
Read newest app log	<code>`sudo cat /var/log/shiny-server/\$(ls -t /var/log/shiny-server/</code>
Generate report	<code>Rscript ~/path/analyse_usage.R ~/Desktop/tier_usage.sqlite</code>

Notes

- All commands assume the server is at `204.168.166.9` and you SSH as `root`.
- All `sudo` commands need to be run on the server, not your Mac.
- Replace `~/path/to/...` with the actual local paths where you store your files.
- Commands marked ⚠ DESTRUCTIVE delete data without confirmation. Backup first.
- Restarting shiny-server kills any active user sessions. Schedule for low-usage times if possible.